Project Title: Lunk Gymness Database

Website link: http://classwork.engr.oregonstate.edu:1342/

1. **Members: Carter Deal, Samuel Dressel**

   Team Name: Team 121

Table of Contents

A) Executive Summary 2
B) Overview 3
C) Database Outline 3
D) ERD Diagram 6
E) Schema Diagram (2NF) 7
F) Sample Data 8
G) Screenshots of the UI: 10

A) Executive Summary

We got rid of the ClassEquipment table to make the relationship between Class and Equipment be 1:M, instead of M:M. Made it so class capacity is dependent on the lowest capacity associated with className and roomNumber. These changes were made based on teacher suggestions.

Over the course of our project, we have added and removed many attributes in tables. We removed specialties from trainers and made it into a biography, which allowed for more characters. We also added a PK, classType, and registrationDate to classRegistrations. For certain attributes, we fixed their naming in code, diagrams, and attributes to fit the camel case project requirements. We made these changes based on teachers' and other students' feedback.

We had a lot of confusion regarding what to display in the ERD diagram vs. the Schema Diagram. The ERD diagram went from just displaying PKs and FKs to displaying FKs, PKs, and significant attributes of tables that would be relevant to our project description and help our gym managers. These changes were based on teacher feedback and student suggestions.

In our UI, we changed how our FKs were displayed by making them be displayed by name instead of ID. When selecting attributes to edit in certain tables, the form entry format went from text-based to dropdown selection. Primary keys can no longer be entered manually into tables via any of the forms. We changed a lot of attribute selection in the edit and add forms to be drop-down menus to meet project requirements and prevent bad data entry. These changes were based on teacher and student comments.

We also used AI tools to assist with this project, especially the built-in Microsoft Copilot and AI-powered auto-completion in VS Code. AI was mainly used to create a Handlebars template based on our database schema. This greatly reduced development time and enabled us to prioritize our work on the SQL code of the database, which we viewed as a strength of its use. A weakness, however, was that it would generate much unnecessary and excessive code.

B) Overview

Lunk Gymness, a newly created gym, has not yet created a digital management system for its gym. So far, they have used analog systems, but that proved to be more demanding as the gym grew. It became difficult to track members on paper, keep track of equipment as more was bought, and handle the creation and scheduling of classes. The website that we are proposing seeks to solve this management problem, creating a backend database of Members, Trainers, Equipment, and Classes, and creating a framework to handle the growth of said entities as the gym inevitably increases in popularity. As a newly created gym, they currently have 100 active members and 10 trainers. The gym has dozens of different types of equipment and hosts classes every day. Classes have a capacity of 15 people. A database-driven website will record class schedules and popularity, equipment usage (for classes and members), and the number of members, as well as monitor the constraints of classes, such as only one trainer, class capacity not being exceeded, and members not registering for the same class twice. Our database is 2NF.

C) Database Outline

- Members: Stores personal and membership information for each gym member.
    - memberID: int, auto_increment, unique, not NULL, PK
    - firstName : varchar(50), not NULL
    - lastName: varchar(50), not NULL
    - email: varchar(100), unique, not NULL
    - phone: varchar(20), NULL
    - membershipType: varchar(20), not NULL (e.g., ‘Basic’, ‘Premium’)
    - joinDate: date, not NULL
    - Relationships:
        - M:M relationship with Classes via ClassRegistrations.

        - 1:M relationship with ClassRegistrations (one member can have many registrations)

- Trainers: Tracks trainers who lead classes and their areas of expertise
    - trainerID: int, auto_increment, unique, not NULL, PK
    - firstName: varchar(50), not NULL
    - lastName: varchar(50), not NULL
    - biography: varchar(255), NULL
    - Relationships:
        - 1:M relationship with Classes (each class has one trainer)
- Classes: Stores details about each class, including schedule and assigned trainer.
    - classID: int, auto_increment, unique, not NULL, PK
    - className: varchar(100), not NULL
    - scheduleTime: datetime, not NULL
    - roomNumber: varchar(10), NULL
    - capacity: int, not NULL,

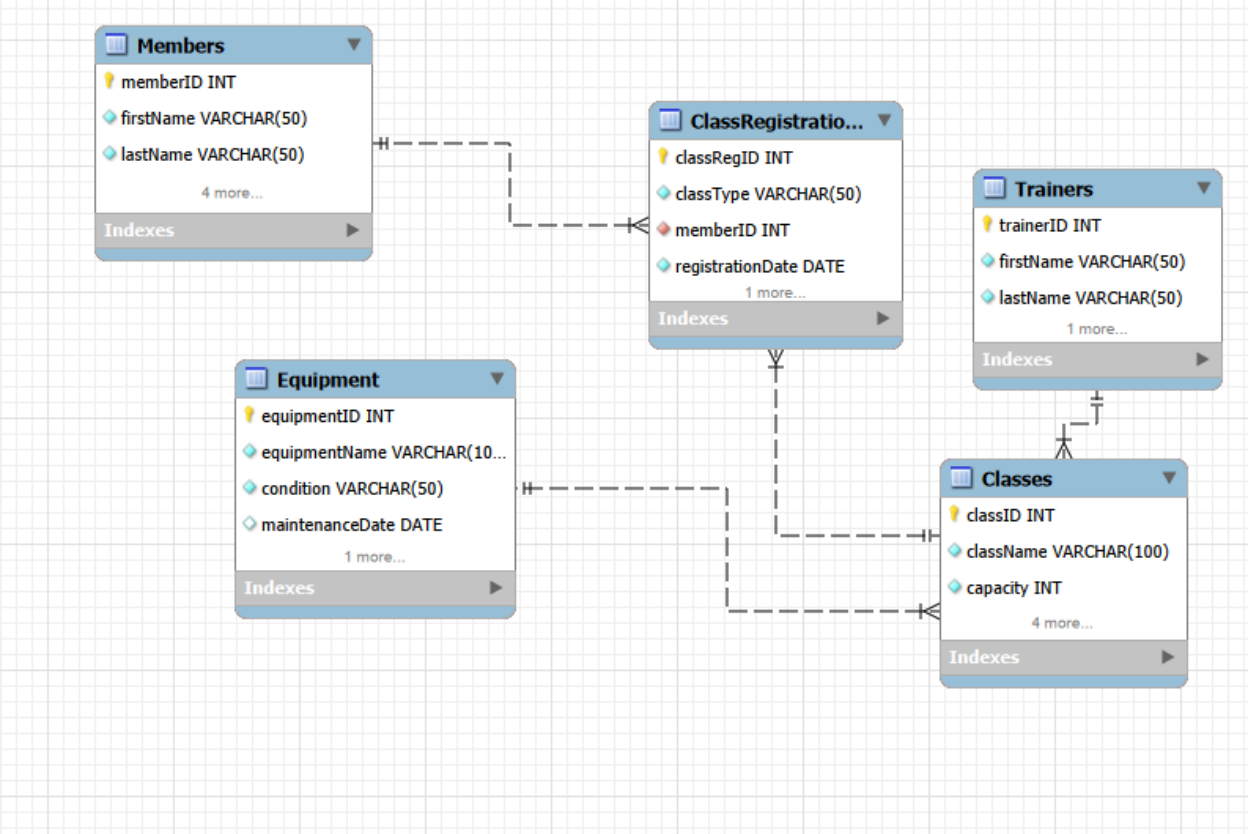
      (Class capacity is dependent on the lowest capacity associated with className and roomNumber. The capacities are as follows:

        - Room 101 capacity = 10
        - Room 102 capacity = 15
        - Room 103 capacity = 20
        - Room 104 capacity = 25
        - Yoga capacity (morning and evening) = 25
        - HIIT Blast capacity = 15
        - Strength training capacity = 10

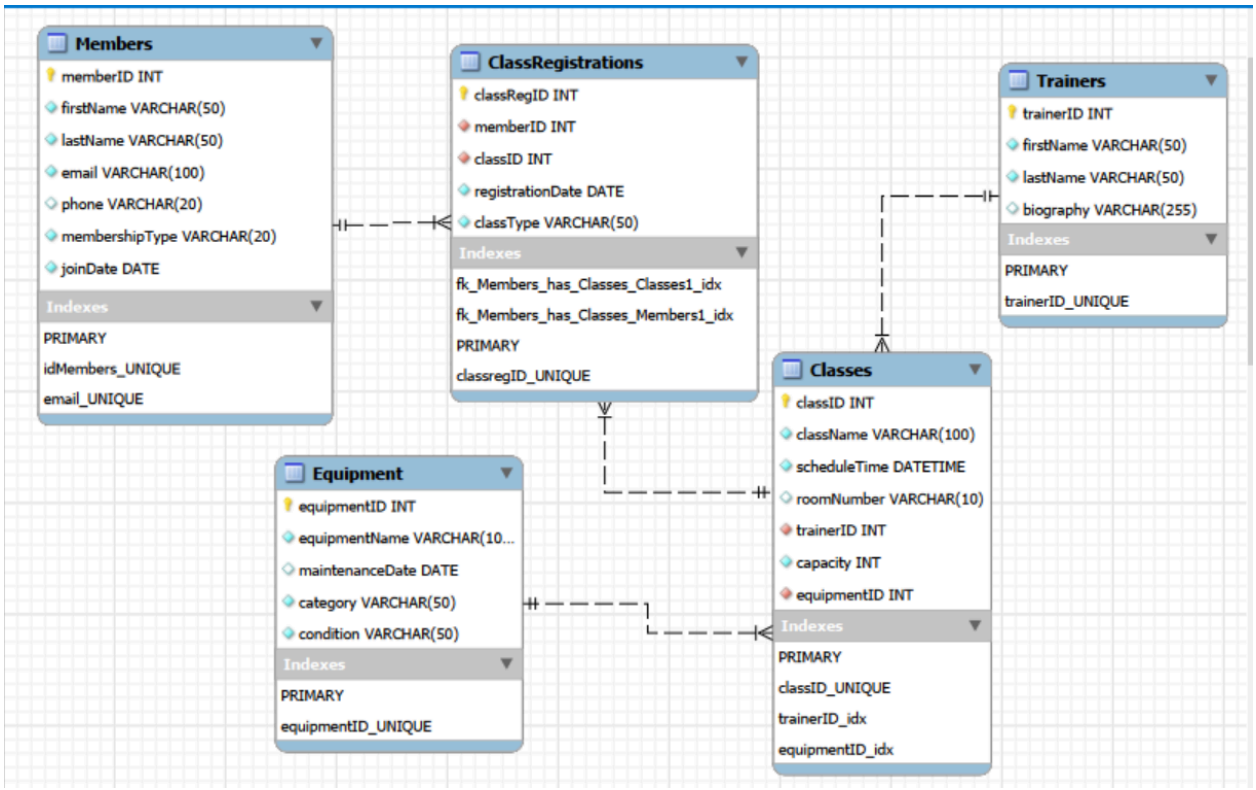
)
    - trainerID: int, not NULL, FK -> Trainers(trainerID)
    - equipmentID: int, not NULL, FK-> Equipment(equipmentID)
    - Relationships:
        - M:1 relationship with Trainers implemented with trainerID as an FK in Classes

        - 1:M relationship with ClassRegistrations
        - M:M relationship with Members via ClassRegistrations.
        - M:1 relationship with Equipment with equipmentID as an FK in classes
- Equipment: Maintains an inventory of equipment and its condition for scheduling and maintenance
    - equipmentID: int, auto_increment, unique, not NULL, PK
    - equipmentName: varchar(100), not NULL
    - category: varchar(50), not NULL (e.g., ‘Cardio’, ‘Strength’
    - maintenanceDate: date, NULL
    - condition: varchar(50), not NULL (e.g., ‘Good’, ‘Needs Repair’)
    - Relationships:
        - 1:M relationship with Classes with equipmentID as an FK in classes
- ClassRegistrations: Intersection table that implements the many-to-many relationship between Members and Classes. Tracks which members are registered for which classes.
    - classRegID: int, auto_increment, unique, not NULL, PK
    - memberID: int, not NULL, FK -> Members(memberID)
    - classID: int, not NULL, FK -> Classes(classID)
    - registrationDate: date, not NULL
    - classType: varchar(50), not NULL
    - Relationships:
        - M:1 relationship with Members with memberID as an FK in ClassRegistrations
        - M:1 relationship with Classes with classID as an FK in ClassRegistrations
        - M:M relationship between Members and Classes (each row represents one member enrolled in one class)

D) ERD Diagram



E) Schema Diagram (2NF)



F) Sample Data

Members

memberID	firstName	lastName	email	phone	membershipType	joinDate
1	Sarah	Kim	sarah.kim@example.com	990-555-2011	Premium	2024-01-15
2	Jason	Patel	jason.patel@example.com	991-555-8890	Basic	2024-02-10
3	Maria	Lopez	maria.lopez@example.com	992-555-4421	Premium	2024-03-05
4	Daniel	Brooks	daniel.brooks@example.com	NULL	Basic	2024-04-01

Trainers

trainerID	firstName	lastName	biography
1	Emily	Chen	Yoga instructor with 5 years of experience
2	Marcus	Reed	Strength training specialist
3	Olivia	Turner	Cardio coach with a focus on endurance

Classes

classID	className	scheduleTime	roomNumber	capacity	trainerID	equipmentID
1	Morning Yoga	2024-05-01 08:00:00	101	10	1	1
2	HIIT Blast	2024-05-01 10:00:00	102	15	3	2
3	Strength Training	2024-05-02 09:30:00	103	10	2	3
4	Evening Yoga	2024-05-02 18:00:00	104	25	1	1

ClassRegistrations

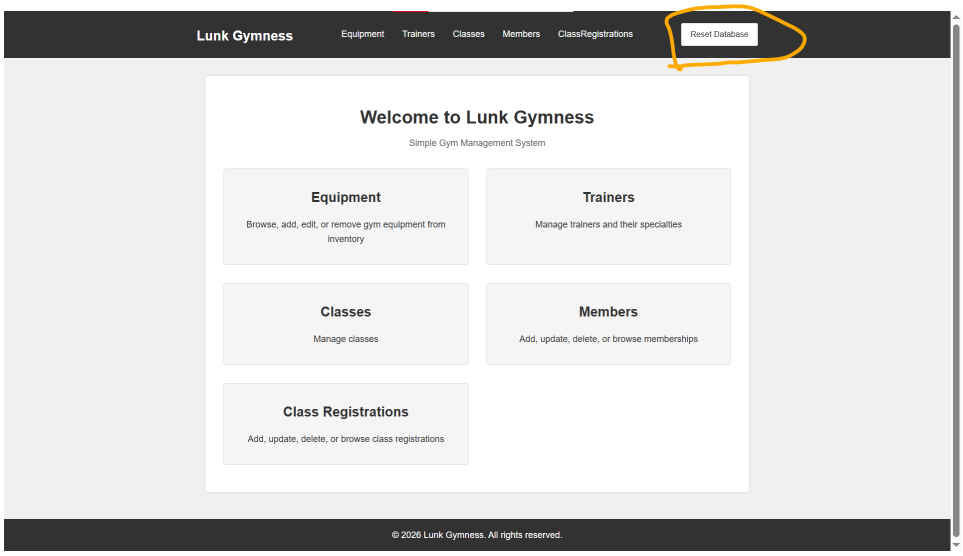
classRegID	memberID	classID	registrationDate	classType
1	1	1	2024-04-20	Yoga
2	2	2	2024-04-22	Cardio
3	3	3	2024-04-25	Strength
4	1	4	2024-04-28	Yoga
5	4	2	2024-04-30	Cardio

Equipment

equipmentID	equipmentName	category	maintenanceDate	condition
1	Yoga Mats	Flexibility	2024-04-10	Good
2	Treadmill	Cardio	2024-03-20	Needs Repair
3	Dumbbells Set	Strength	2024-04-15	Good
4	Rowing Machine	Cardio	2024-02-28	Good

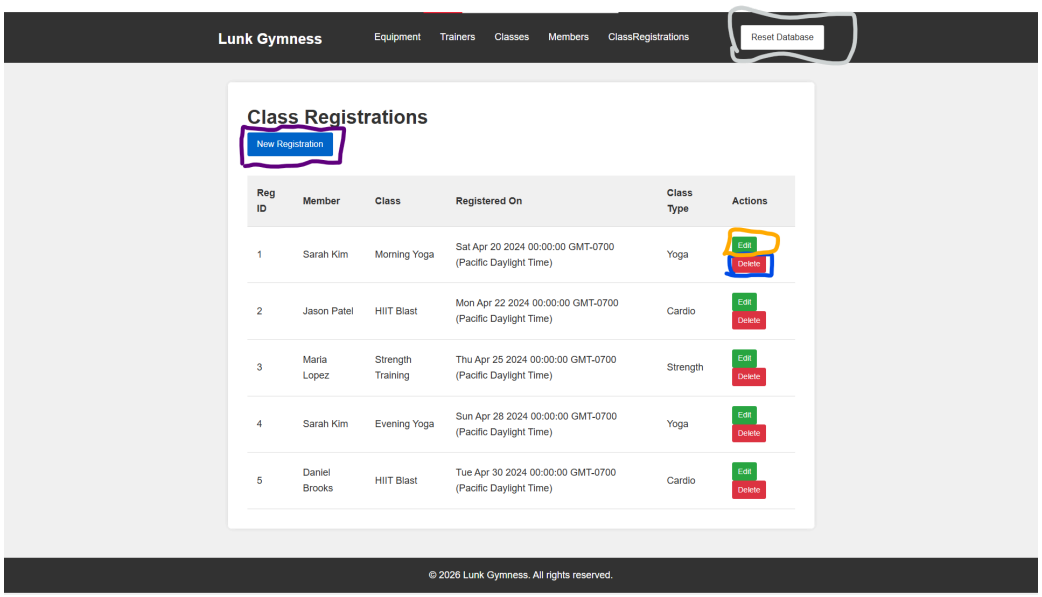
G) Screenshots of the UI:

Lunk Gymness Home Page



This is the lunk gymness homepage which links to each other page in the UI. In the top of the navbar on the top right of the website is the reset database button. The reset button is circled in orange. This button resets the database.

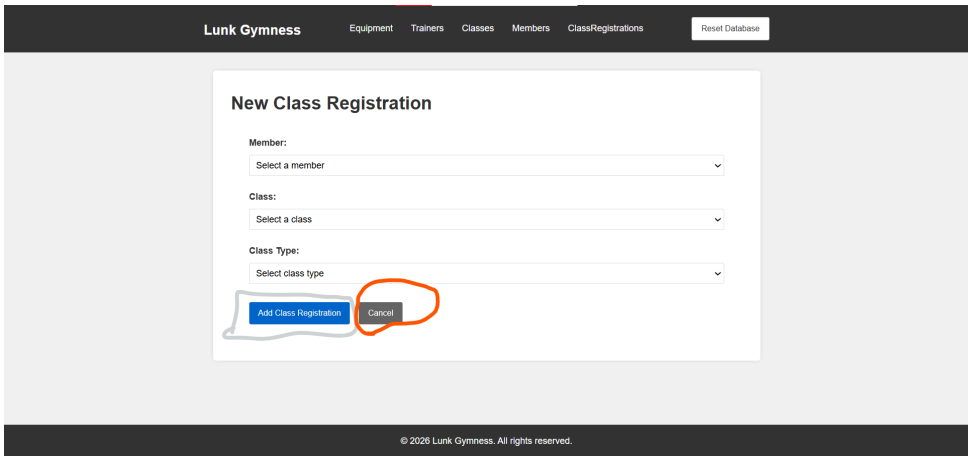
Class Registrations (M:M) Select/Read/Browse/Delete Page



Above is the picture of the M:M table in our database, known ClassRegistrations. This page is the select, read, browse, and delete page for our M:M table. If you click the red delete button circled in orange, you will delete the selected row in the table after being prompted to make sure you want to delete that row. If you click the edit button circled in orange, you will be brought to the edit/update ClassRegistration form. If you click on the new registration button circled in purple, you will be brought to add a new registration form. This type of logic and layout is consistent throughout the database. If you click the reset database

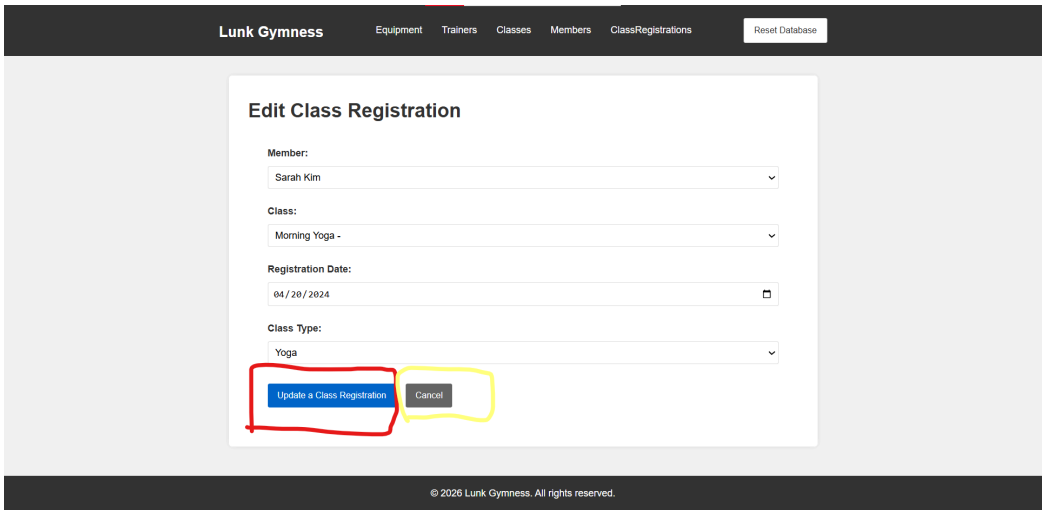
button circled in grey, you will reset the database. If you click on any of the page names at the top of the site in the navbar, you will be brought to that page on the site.

Class Registrations (M:M) Add/Create Form Page



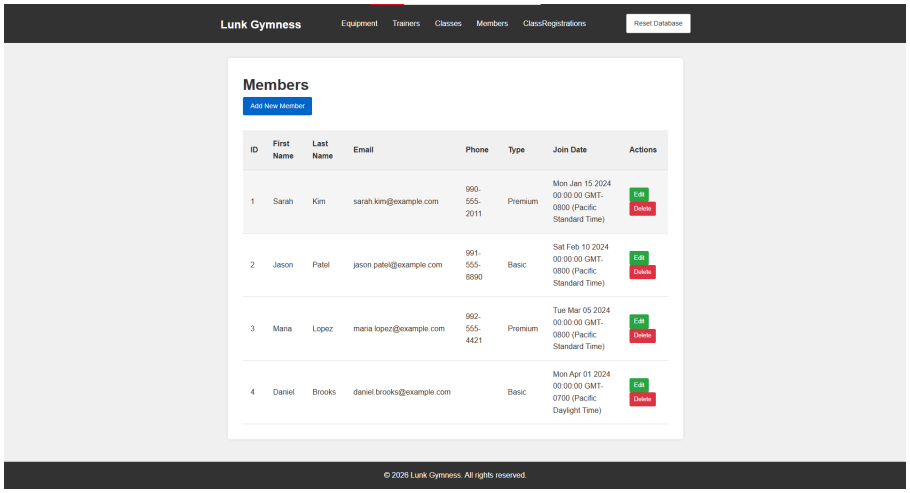
Above is the add/create form for our M:M table/relationship for class registrations. If you click the add class registration button circled in grey you will add a new class registration to the class registration table. If you click the cancel button circled in red you will be brought back to the select, read, browse, and delete for class registrations and cancel whatever class registration you were about to add. This type of form logic and button logic for creating new rows in tables is consistent across the entire site.

Class Registrations (M:M) Edit/Update From Page

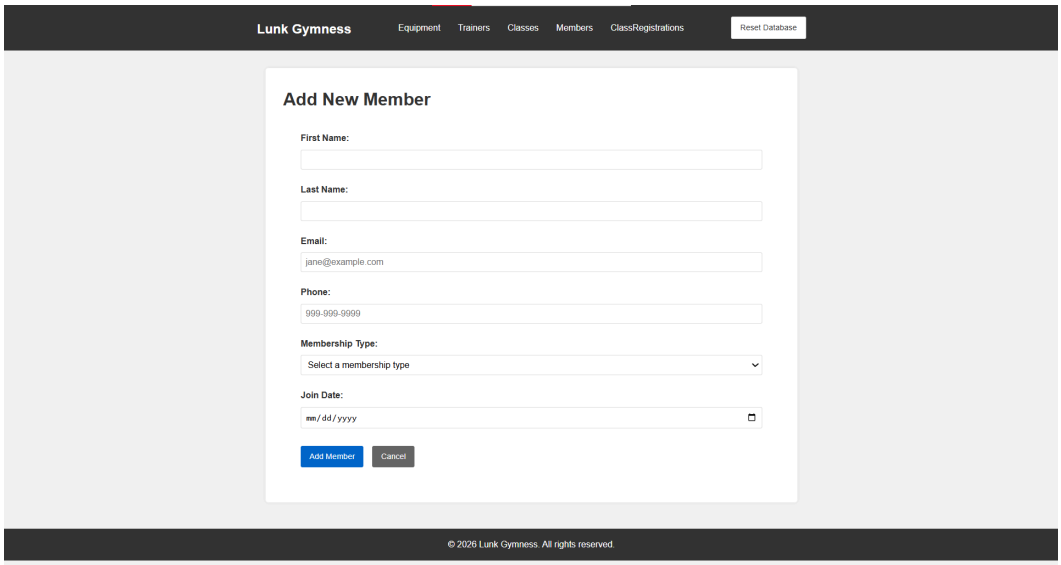


Above is the M:M edit form for class registrations. If you click the update a class registration button, which is circled in red, you will update the row you have selected in class registrations with whatever edits you have made. If you click the cancel button circled in yellow, you will cancel the row update you were going to and be brought back to the class registrations browse, select, read, and delete page.

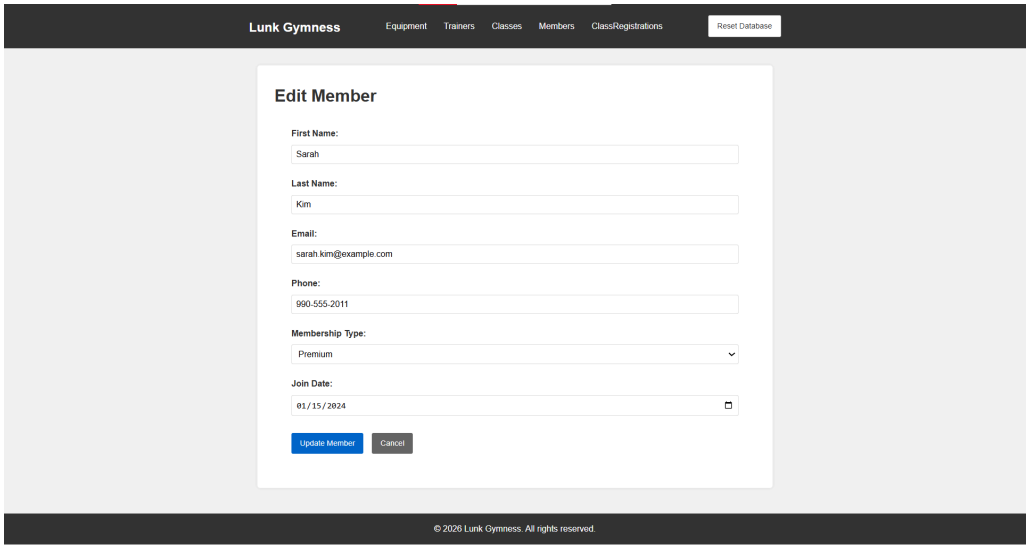
Members Select/Read/Browse/Delete Page



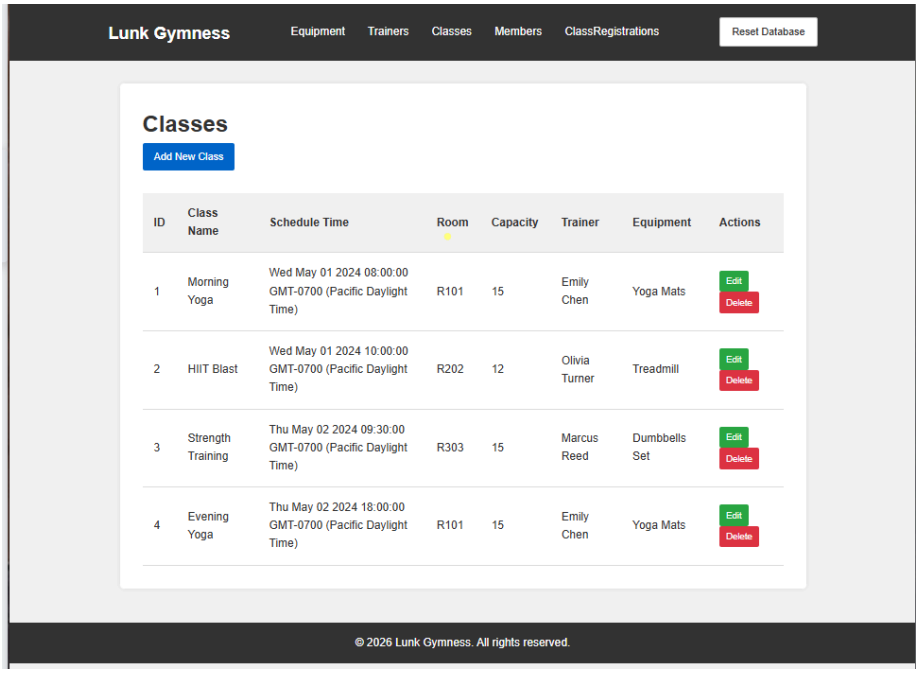
Members Add/Create Form Page



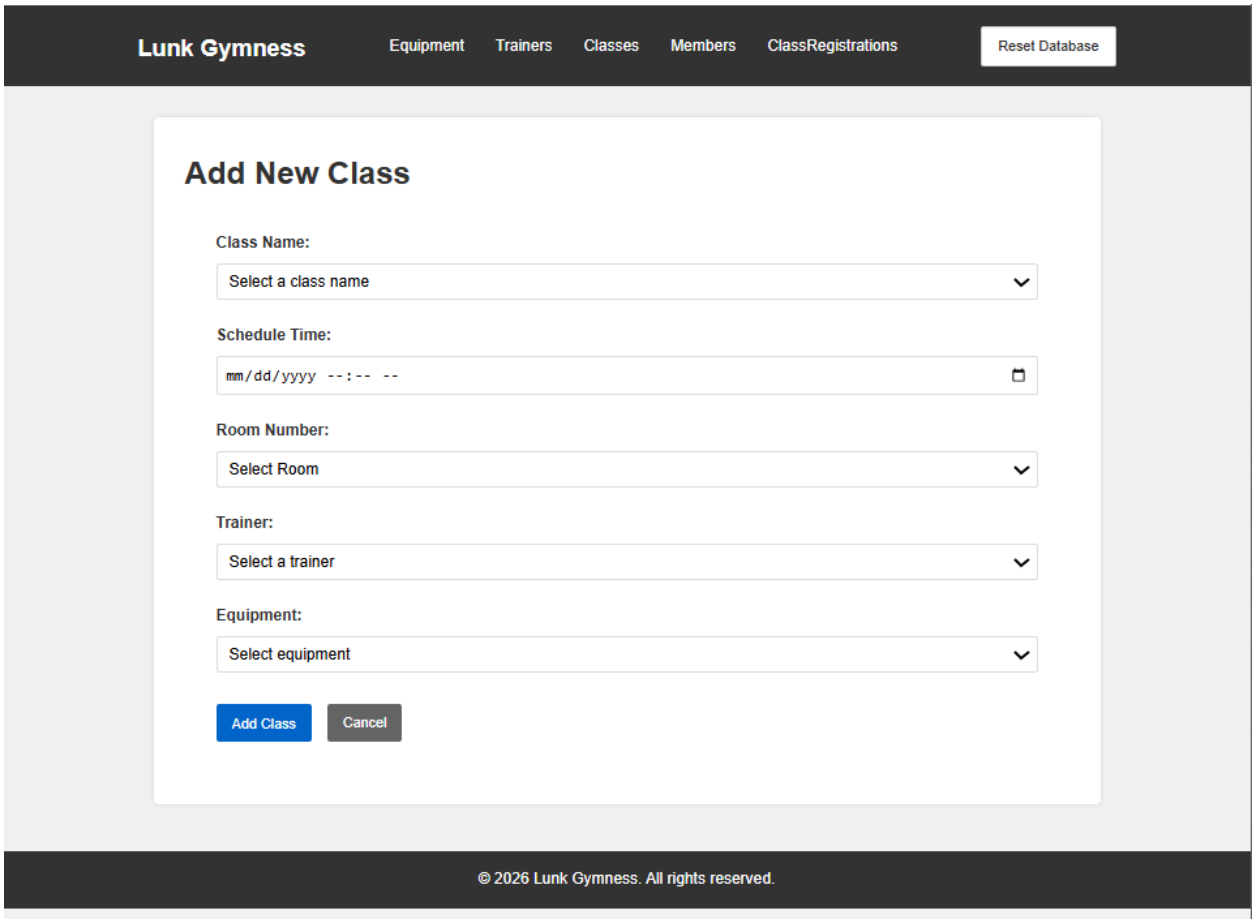
Members Edit/Update Form Page



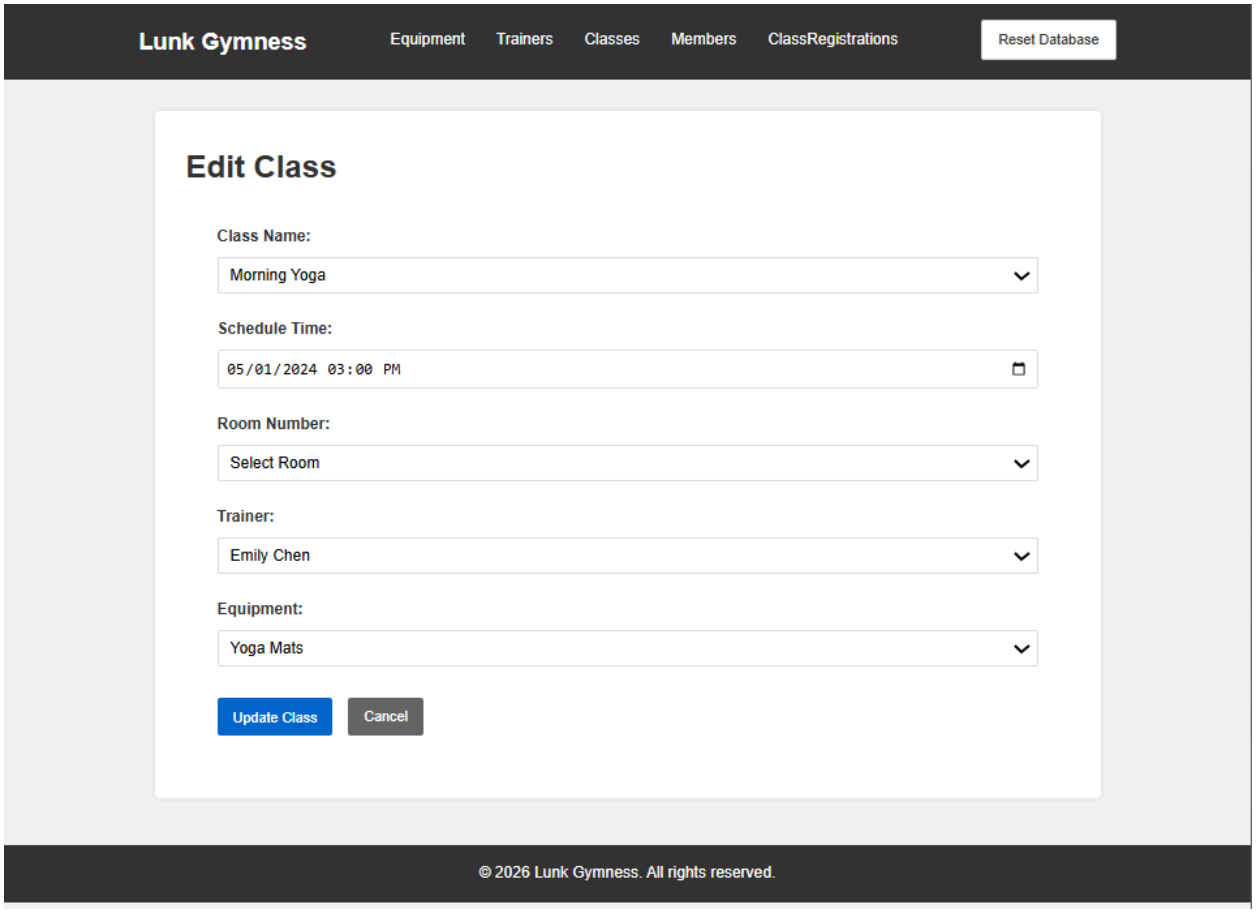
Classes Select/Read/Browse/Delete Page



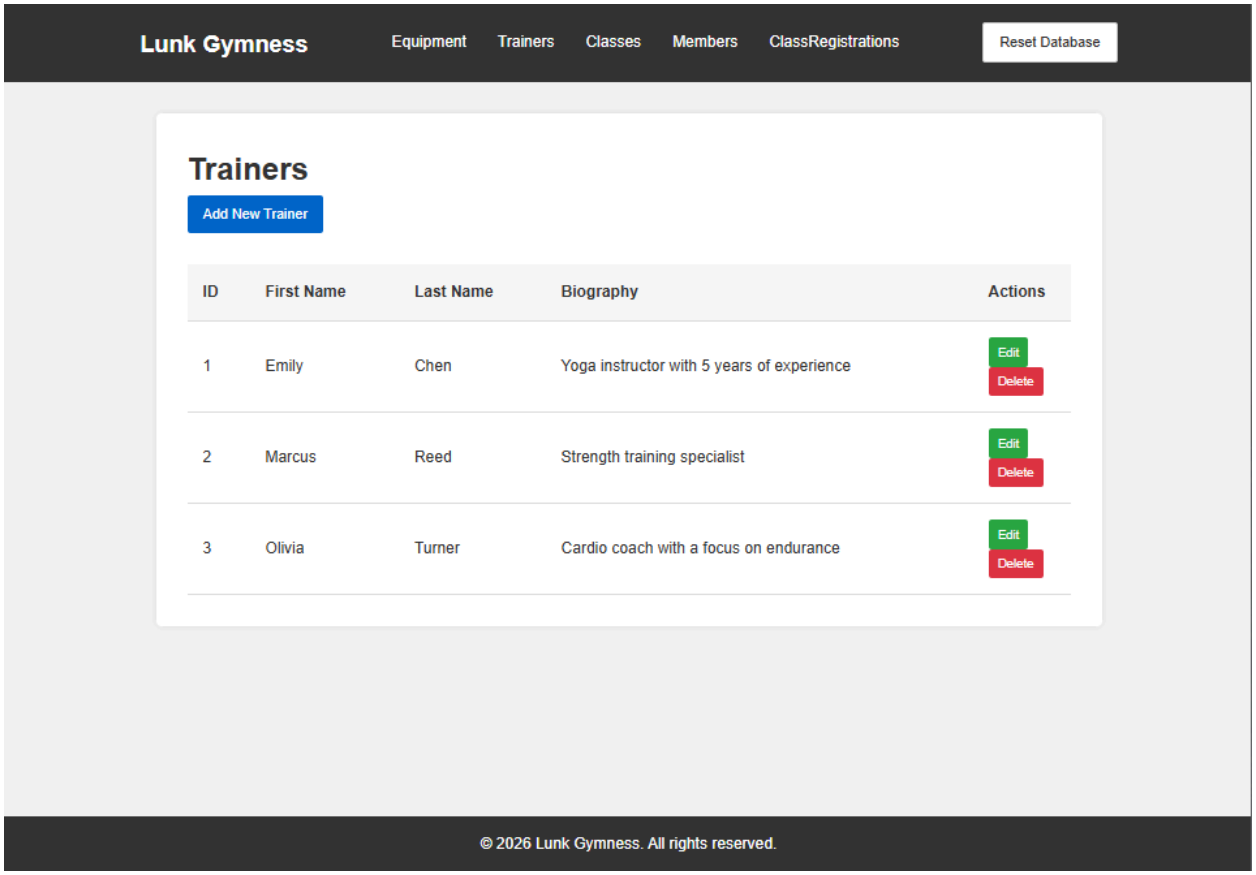
Classes Add/Create Form Page



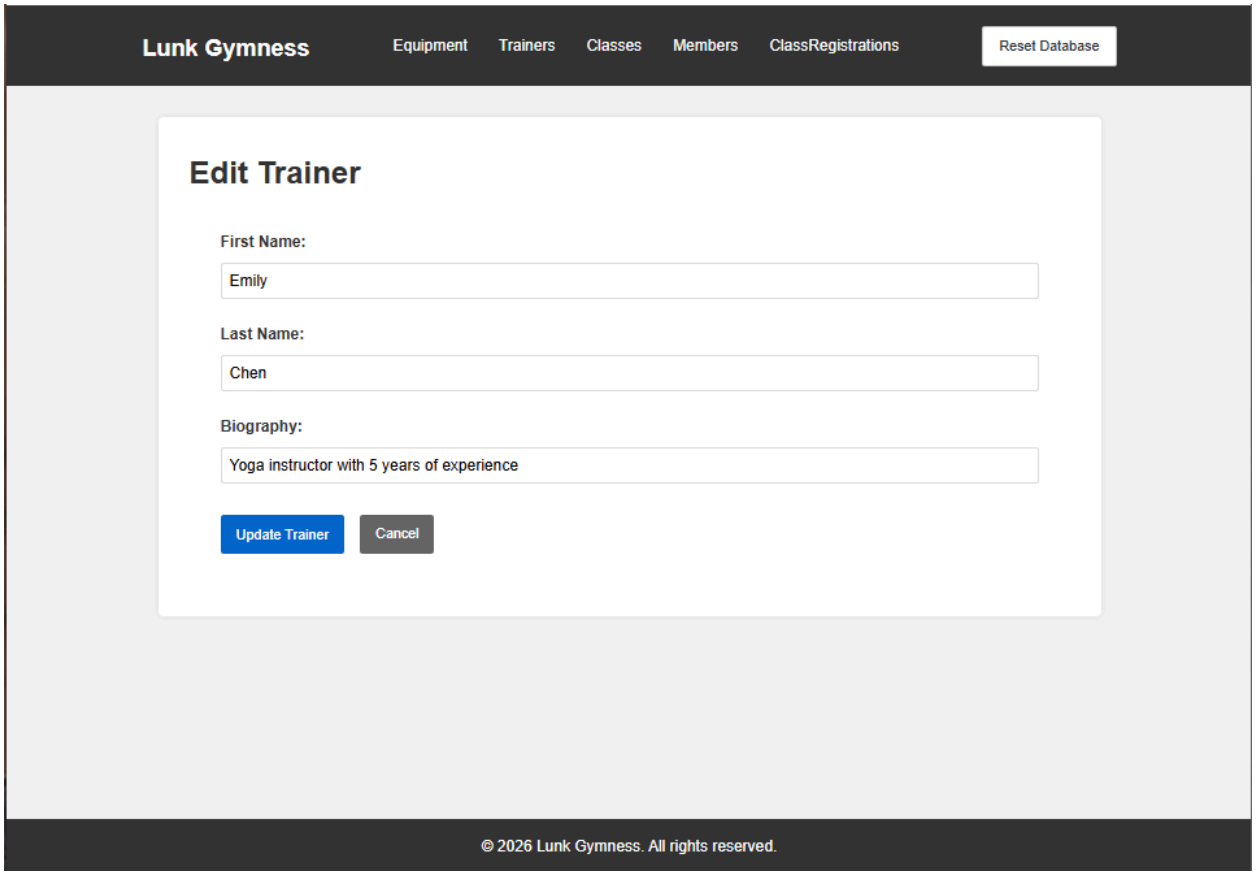
Classes Edit/Update Form Page



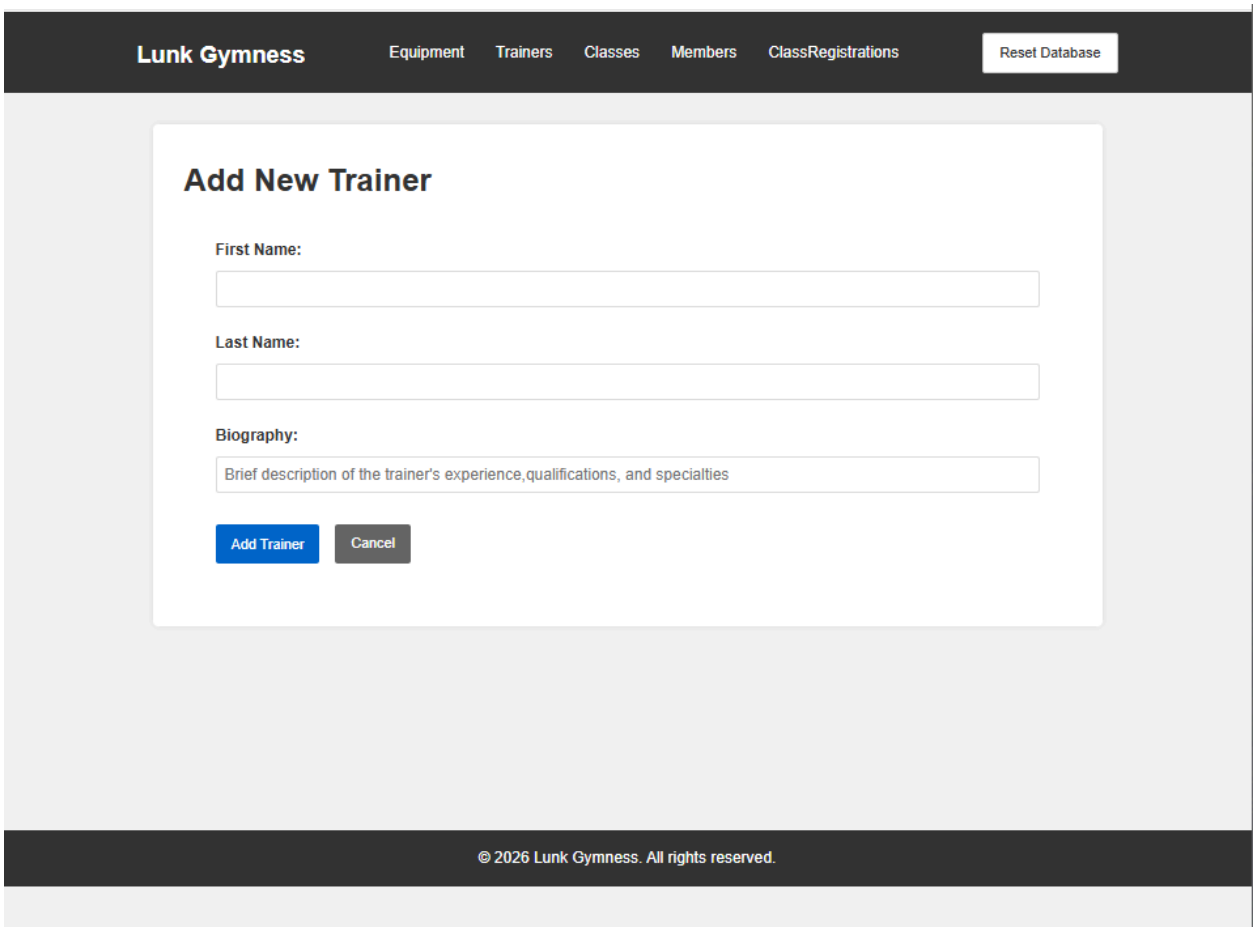
Trainers Select/Read/Browse/Delete Form Page



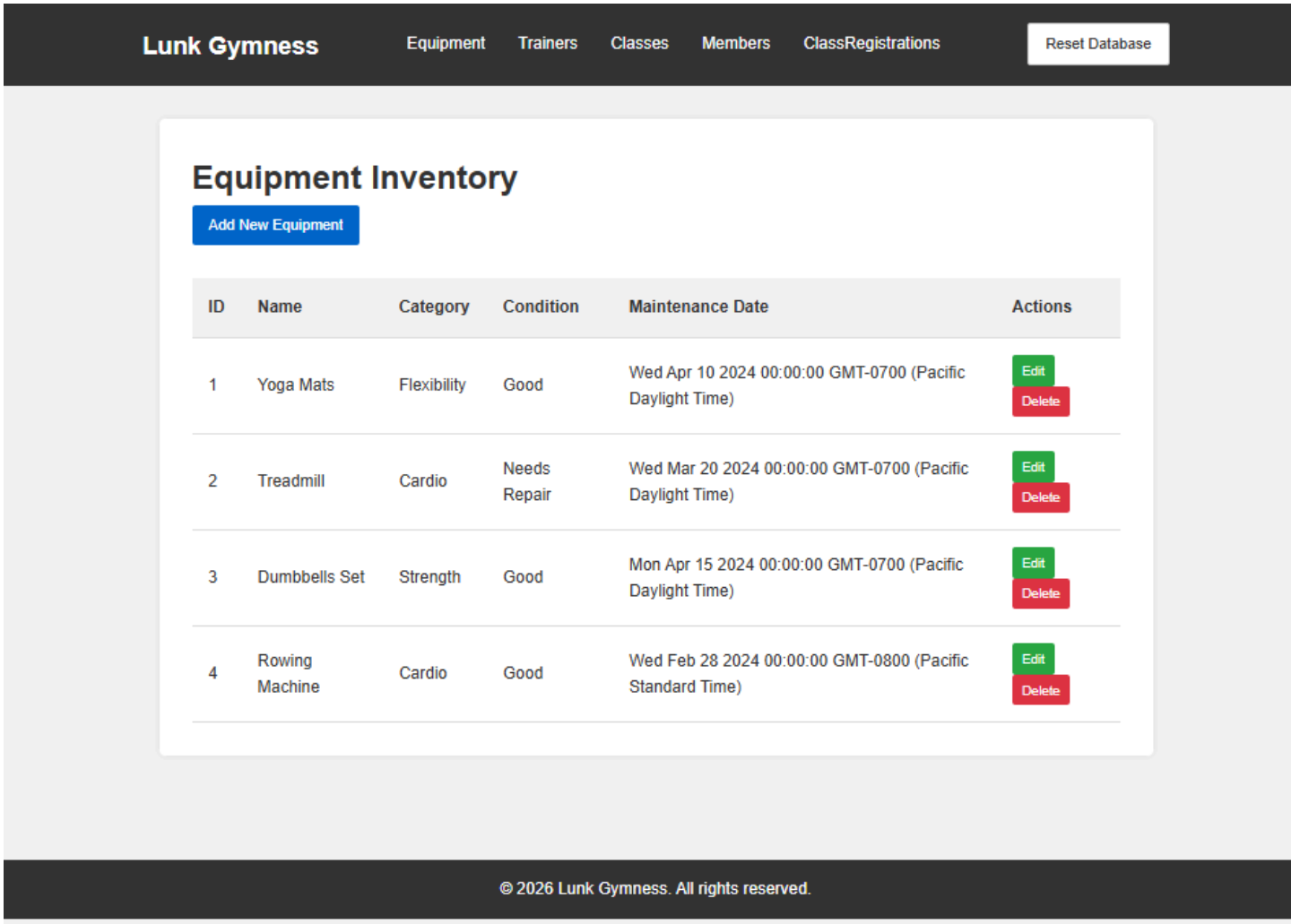
Trainers Edit/Update Form Page



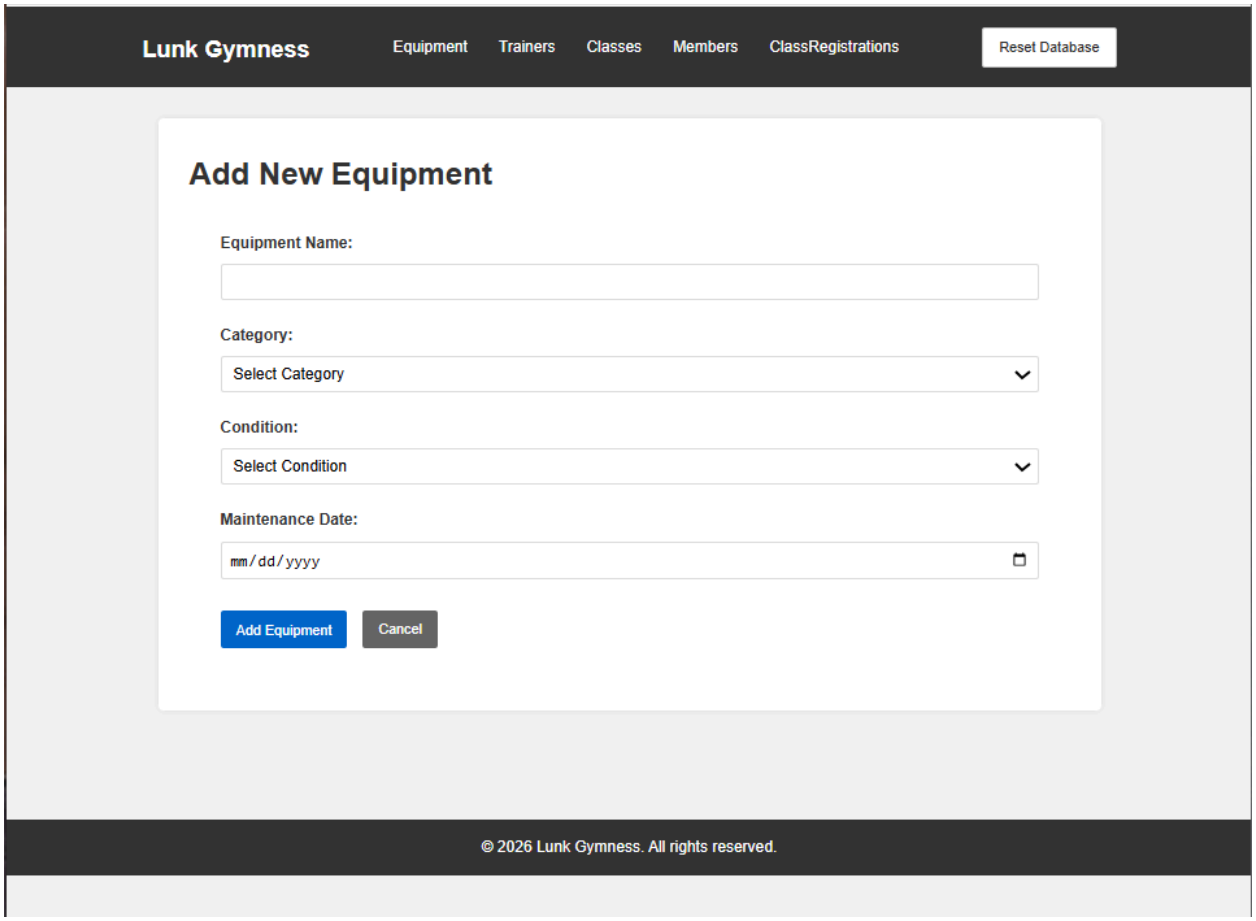
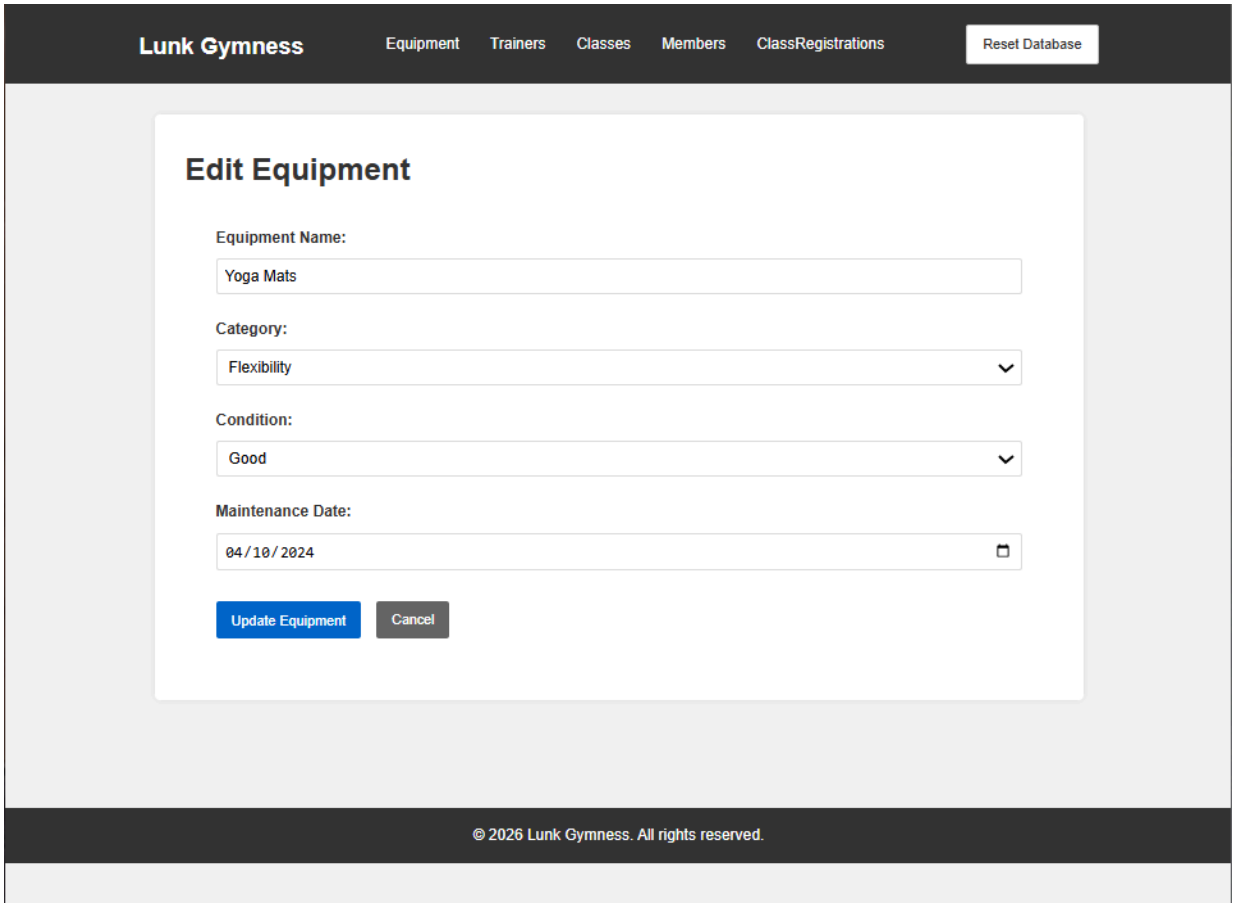
Trainers Add/Create Form Page



Equipment Read/Browse/Select/Delete Page



Equipment Add/Create Form Page



Equipment Edit/Update Form